F-Praktikum: Simulating a Quantum Computer

Theoretical Foundations and Software Implementation

Quantum Simulation Project

April 28, 2026

Abstract

This report details the theoretical foundations of quantum computation and their programmatic implementation using Python's `qutip` and `qutip_qip` libraries. The codebase is structured across five modules: `quantum_gates.py` (gate definitions and utilities), `simulate.py` (single-qubit gate simulation), `bloch_sphere.py` (visualisation), `bell_state.py` (two-qubit entanglement), and `gate_test.py` (automated unit tests). The report covers qubits and state vectors, unitary gate operations, multi-qubit tensor products, the Bloch sphere representation, and entangled Bell states, connecting each concept explicitly to the corresponding Python implementation.

Contents

1 Foundations of Quantum Information

To build a quantum simulator, physical quantum phenomena must be mapped onto computable mathematical structures. The following core concepts form the basis of the simulation.

1.1 Qubits and State Vectors

Unlike classical bits, which are strictly deterministic (0 or 1), a quantum bit (qubit) exists in a two-dimensional complex vector space (a Hilbert space). The fundamental computational basis states are written in Dirac notation:

$$|0\rangle = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \quad |1\rangle = \begin{pmatrix} 0 \\ 1 \end{pmatrix} \tag{1}$$

A general single-qubit state is a superposition of these bases,

$$|\psi\rangle = \alpha\,|0\rangle + \beta\,|1\rangle\,, \qquad |\alpha|^2 + |\beta|^2 = 1, \tag{2}$$

where $\alpha, \beta \in \mathbb{C}$ are probability amplitudes. The normalisation condition ensures that measuring the qubit yields a well-defined probability for each outcome.

Implementation (`quantum_gates.py`, `bell_state.py`). QuTiP represents state vectors as `Qobj` column vectors. Basis states are created with `qt.basis(N, m)`, where $N = 2$ for a qubit and $m \in \{0, 1\}$ selects the level:

```
import qutip as qt

ket0 = qt.basis(2, 0)    # |0> = [1, 0]^T
ket1 = qt.basis(2, 1)    # |1> = [0, 1]^T
```

Listing 1: Basis state initialisation (`quantum_gates.py` / `bell_state.py`)

A utility function `get_state_vector` flattens the QuTiP object back into a standard NumPy array for inspection and numerical testing:

```
import numpy as np

def get_state_vector(state: qt.Qobj) -> np.ndarray:
    """Reads out the state vector as a standard 1D numpy array."""
    return state.full().flatten()
```

Listing 2: `get_state_vector` utility (`quantum_gates.py`)

1.2 The Bloch Sphere Representation

Any pure single-qubit state can be parametrised uniquely by two angles $\theta \in [0, \pi]$ and $\phi \in [0, 2\pi)$ on the surface of the unit sphere (the *Bloch sphere*):

$$|\psi\rangle = \cos\tfrac{\theta}{2}\,|0\rangle + e^{i\phi}\sin\tfrac{\theta}{2}\,|1\rangle\,. \tag{3}$$

The corresponding Bloch vector $\mathbf{r} = (x, y, z)$ is obtained from the expectation values of the three Pauli operators:

$$x = \langle\sigma_x\rangle, \quad y = \langle\sigma_y\rangle, \quad z = \langle\sigma_z\rangle. \tag{4}$$

The north pole $|0\rangle$ has $\mathbf{r} = (0, 0, 1)$ and the south pole $|1\rangle$ has $\mathbf{r} = (0, 0, -1)$.

Implementation (`bloch_sphere.py`). The function `get_bloch_coords` computes the Bloch vector directly from QuTiP's `qt.expect` together with the built-in Pauli operators:

```python
def get_bloch_coords(state: qt.Qobj) -> tuple[float, float, float]:
    """Calculate (x, y, z) Bloch coordinates via Pauli expectation values."""
    x = qt.expect(qt.sigmax(), state)
    y = qt.expect(qt.sigmay(), state)
    z = qt.expect(qt.sigmaz(), state)
    return float(x), float(y), float(z)
```

Listing 3: Bloch vector computation (`bloch_sphere.py`)

This is called for every gate output in `simulate.py` to populate the printed table of Bloch coordinates and to position state vectors on the rendered sphere.

2 Quantum Gates as Unitary Operators

In quantum mechanics, closed-system time evolution is governed by unitary transformations: a gate U must satisfy $U^\dagger U = I$, ensuring reversibility and probability conservation. Applying a gate to a state is matrix–vector multiplication, $|\psi'\rangle = U\,|\psi\rangle$.

2.1 Standard Single-Qubit Gates

The gates implemented in `quantum_gates.py` cover the standard set from Nielsen & Chuang (Chapter 4.2):

$$X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \quad Y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}, \quad Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} \tag{5}$$

$$H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}, \quad S = \begin{pmatrix} 1 & 0 \\ 0 & i \end{pmatrix}, \quad T = \begin{pmatrix} 1 & 0 \\ 0 & e^{i\pi/4} \end{pmatrix} \tag{6}$$

The Pauli matrices X, Y, Z are available directly from QuTiP. The Hadamard gate H is constructed via `hadamard_transform()`, while S and T are obtained from the parametrised phase gate $P(\phi) = \mathrm{diag}(1, e^{i\phi})$:

```python
import qutip as qt
from qutip_qip.operations import hadamard_transform, phasegate

X = qt.sigmax()
Y = qt.sigmay()
Z = qt.sigmaz()

H = hadamard_transform()
S = phasegate(np.pi / 2)    # S = Z^(1/2)
T = phasegate(np.pi / 4)    # T = Z^(1/4),  the pi/8 gate
```

Listing 4: Standard gate definitions (`quantum_gates.py`)

2.2 Rotation Gates

Continuous rotations about each Bloch-sphere axis are parametrised by an angle θ:

$$R_x(\theta) = e^{-i\theta X/2} = \begin{pmatrix} \cos\frac{\theta}{2} & -i\sin\frac{\theta}{2} \\ -i\sin\frac{\theta}{2} & \cos\frac{\theta}{2} \end{pmatrix} \tag{7}$$

and analogously for R_y and R_z. These are wrappers around `qutip_qip`'s `rx`, `ry`, `rz`:

```python
from qutip_qip.operations import rx, ry, rz

def Rx(theta: float) -> qt.Qobj:
    """Rotation by theta about the x-axis of the Bloch sphere."""
    return rx(theta)

def Ry(theta: float) -> qt.Qobj:
    return ry(theta)

def Rz(theta: float) -> qt.Qobj:
    return rz(theta)
```

Listing 5: Rotation gate wrappers (`quantum_gates.py`)

2.3 Unitarity Verification

QuTiP's `Qobj.isunitary` property performs the check $U^\dagger U = I$ numerically. The helper `is_unitary` exposes this for every gate in the simulation:

```python
def is_unitary(U: qt.Qobj) -> bool:
    return U.isunitary
```

Listing 6: Unitarity check (`quantum_gates.py`)

In `simulate.py` this is called over all nine gates after the gate application loop, printing a verification table to standard output.

2.4 Applying Gates to States

Because quantum logic is matrix multiplication, a thin `apply` wrapper makes the intent explicit and keeps calling code readable:

```python
def apply(gate: qt.Qobj, state: qt.Qobj) -> qt.Qobj:
    """Applies a quantum gate to a state via matrix
   multiplication."""
    return gate * state
```

Listing 7: `apply` wrapper (`quantum_gates.py`)

3 The Single-Qubit Simulation Pipeline (`simulate.py`)

`simulate.py` ties together the gate and visualisation modules into a complete single-qubit experiment. Starting from the south-pole state $|1\rangle$, it applies every standard and rotation gate, prints the resulting amplitudes and Bloch coordinates, verifies unitarity, and produces three figures.

```
THETA = np.pi / 2    # angle for rotation gates

gate_specs = [
    ("X  (Pauli-X)",  X,         "#e74c3c"),
    ("Y  (Pauli-Y)",  Y,         "#9b59b6"),
    ("Z  (Pauli-Z)",  Z,         "#2980b9"),
    ("H  (Hadamard)", H,         "#27ae60"),
    ("S  (Phase)",    S,         "#f39c12"),
    ("T  (pi/8)",     T,         "#16a085"),
    ("Rx(pi/2)",      Rx(THETA),"#c0392b"),
    ("Ry(pi/2)",      Ry(THETA),"#8e44ad"),
    ("Rz(pi/2)",      Rz(THETA),"#1abc9c"),
]

for label, gate, color in gate_specs:
    out   = apply(gate, ket1)
    alpha = out.full()[0, 0]    # amplitude of |0>
    beta  = out.full()[1, 0]    # amplitude of |1>
    bx, by, bz = get_bloch_coords(out)
    entries.append((label, out, color))
```

Listing 8: Main simulation loop (`simulate.py`)

Table 1 summarises the expected action of each gate on $|1\rangle$.

Table 1: Action of single-qubit gates on $|1\rangle$, as produced by `simulate.py`.

Gate	**Output state**	**Bloch vector** (x, y, z)
X	$\|0\rangle$	$(0, 0, +1)$
Y	$i\|0\rangle$	$(0, 0, +1)$
Z	$-\|1\rangle$	$(0, 0, -1)$
H	$(\|0\rangle - \|1\rangle)/\sqrt{2}$	$(-1, 0, 0)$
S	$-i\|1\rangle$	$(0, 0, -1)$
T	$e^{-i\pi/4}\|1\rangle$	$(0, 0, -1)$
$R_x(\pi/2)$	$(\|0\rangle - i\|1\rangle)/\sqrt{2}$	$(0, -1, 0)$
$R_y(\pi/2)$	$(-\|0\rangle + \|1\rangle)/\sqrt{2}$	$(-1, 0, 0)$
$R_z(\pi/2)$	$e^{i\pi/4}\|1\rangle$	$(0, 0, -1)$

4 Visualisation on the Bloch Sphere (`bloch_sphere.py`)

`bloch_sphere.py` provides two public plotting functions that `simulate.py` calls after the simulation loop.

4.1 Panel Plot: `plot_bloch_panels`

Each gate output is rendered on its own Bloch sphere subplot. The initial state $|1\rangle$ is shown as a grey reference vector and the gate output as a coloured vector:

```
def plot_bloch_panels(entries, ncols=3, suptitle=..., filename=
    None):
```

```
    for k, (label, state, color) in enumerate(entries, 1):
        b = qt.Bloch(fig=fig, axes=ax)
        b.vector_color = ["#cccccc", color]    # grey = |1>, color = output
        b.add_states(state_1)                   # reference
        b.add_states(state)                     # gate output
        b.render()
```

Listing 9: Panel Bloch sphere plot (`bloch_sphere.py`)

This is called twice in `simulate.py`: once for the six standard gates (`bloch_standard_gates.png`) and once for the three rotation gates (`bloch_rotation_gates.png`).

4.2 Composite Plot: `plot_all_states_single_sphere`

All gate outputs are overlaid on a single sphere, using the per-gate colours defined in the `gate_specs` list. This gives an at-a-glance view of how each gate moves the state vector on the Bloch sphere:

```
def plot_all_states_single_sphere(entries, initial_state, filename=None):
    states = [initial_state] + [s for _, s, _ in entries]
    colors = ["#888888"] + [c for _, _, c in entries]
    b_all.add_states(states)
    b_all.vector_color = colors
    b_all.render()
```

Listing 10: All-states Bloch sphere (`bloch_sphere.py`)

The output is saved as `bloch_all_gates.png`.

5 Multi-Qubit Systems and Tensor Products

5.1 Theory

When combining n qubits, their individual Hilbert spaces are joined by the tensor product $\otimes$. For two qubits A and B:

$$|\psi_{AB}\rangle = |\psi_A\rangle \otimes |\psi_B\rangle \,. \tag{8}$$

The resulting Hilbert space has dimension 2^n. For $n = 2$ the four computational basis states are $\{|00\rangle, |01\rangle, |10\rangle, |11\rangle\}$, represented as a 4×1 column vector with the ordering

$$|00\rangle = \begin{pmatrix}1\\0\\0\\0\end{pmatrix}, \quad |01\rangle = \begin{pmatrix}0\\1\\0\\0\end{pmatrix}, \quad |10\rangle = \begin{pmatrix}0\\0\\1\\0\end{pmatrix}, \quad |11\rangle = \begin{pmatrix}0\\0\\0\\1\end{pmatrix}. \tag{9}$$

5.2 Gate Expansion

A critical challenge is *dimensionality mismatch*: a 2×2 single-qubit gate cannot act directly on a 4×1 two-qubit state. The gate must be embedded into the full n-qubit operator space by tensoring with identity matrices on the untargeted qubits. For Hadamard on qubit 0 in a two-qubit system:

$$H^{(0)} = H \otimes I, \qquad H^{(1)} = I \otimes H. \tag{10}$$

Implementation (`quantum_gates.py`). `apply_single_gate` delegates this expansion to `qutip_qip`'s `expand_operator`:

```
from qutip_qip.operations import expand_operator

def apply_single_gate(gate: qt.Qobj, N: int, target: int) -> qt.
    Qobj:
    """Expands a 1-qubit gate to an N-qubit system."""
    return expand_operator(gate, N=N, targets=[target])
```

Listing 11: Gate expansion to N-qubit space (`quantum_gates.py`)

5.3 Multi-Qubit Gates

Three multi-qubit gates are defined in `quantum_gates.py`:

```
from qutip_qip.operations import cnot, iswap, toffoli

def CNOT(N=2, control=0, target=1) -> qt.Qobj:
    return cnot(N, control, target)

def ISWAP(N=2, targets=[0, 1]) -> qt.Qobj:
    return iswap(N, targets)

def TOFFOLI(N=3, controls=[0, 1], target=2) -> qt.Qobj:
    return toffoli(N, controls, target)
```

Listing 12: Multi-qubit gate wrappers (`quantum_gates.py`)

The **CNOT** (Controlled-NOT) gate flips the target qubit if and only if the control qubit is $|1\rangle$. Its matrix in the computational basis is

$$\mathrm{CNOT} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}. \tag{11}$$

The **iSWAP** gate swaps two qubits and adds a relative phase of i:

$$\mathrm{iSWAP} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & i & 0 \\ 0 & i & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}. \tag{12}$$

The **Toffoli** (CCNOT) gate is a three-qubit gate that flips the target qubit when *both* control qubits are $|1\rangle$; it is the quantum analogue of a classical AND gate.

6 Simulating Entanglement: The Bell State (`bell_state.py`)

6.1 Theory

Quantum entanglement is a non-classical correlation between qubits that cannot be described by any product state. The four maximally entangled two-qubit states are the

Bell states; the target of our simulation is

$$\left|\Phi^{+}\right\rangle = \frac{1}{\sqrt{2}}\Big(|00\rangle + |11\rangle\Big). \tag{13}$$

It is prepared by the circuit $\mathrm{CNOT}_{01} \cdot (H \otimes I)$ acting on $|00\rangle$:

$$|00\rangle \xrightarrow{H\otimes I} \frac{1}{\sqrt{2}}\Big(|00\rangle + |10\rangle\Big) \xrightarrow{\text{CNOT}} \frac{1}{\sqrt{2}}\Big(|00\rangle + |11\rangle\Big). \tag{14}$$

6.2 Implementation (`bell_state.py`)

```
# 1. Initialise |00>
ket0     = qt.basis(2, 0)
psi_init = qt.tensor(ket0, ket0)

# 2. Apply Hadamard to qubit 0  (H must be expanded to H (x) I)
H_expanded = apply_single_gate(H, N=2, target=0)
psi_step1  = apply(H_expanded, psi_init)
# State is now (|00> + |10>) / sqrt(2)

# 3. Apply CNOT (control=0, target=1)
cnot_gate = CNOT(N=2, control=0, target=1)
psi_final = apply(cnot_gate, psi_step1)

# 4. Verification
final_vec = get_state_vector(psi_final)
amp_00    = final_vec[0]   # index 0 <-> |00>
amp_11    = final_vec[3]   # index 3 <-> |11>
# Expected: both amplitudes = 1/sqrt(2) ~ 0.7071
```

Listing 13: Bell state circuit (`bell_state.py`)

The output vector is $[0.707, 0, 0, 0.707]^T$, confirming that the amplitude is distributed exclusively between $|00\rangle$ (index 0) and $|11\rangle$ (index 3), with all other components equal to zero.

7 Automated Unit Testing (`gate_test.py`)

Correctness of the gate and utility implementations is verified by a `unittest` test suite in `gate_test.py`. Four test cases cover state readout and each of the three multi-qubit gates.

```
class TestQuantumSimulation(unittest.TestCase):

    def test_state_vector_readout(self):
        """get_state_vector should return [0, 1] for |1>."""
        vec = get_state_vector(self.ket1)
        np.testing.assert_array_almost_equal(vec, [0.+0.j, 1.+0.j
    ])

    def test_cnot_gate(self):
```

```
        """CNOT on |10> (control=0, target=1) must give |11>."""
        cnot_op    = CNOT(N=2, control=0, target=1)
        output     = apply(cnot_op, self.ket10)
        np.testing.assert_array_almost_equal(
            get_state_vector(output),
            get_state_vector(self.ket11))

    def test_iswap_gate(self):
        """iSWAP on |01> must give i|10>."""
        iswap_op = ISWAP(N=2, targets=[0, 1])
        output   = apply(iswap_op, self.ket01)
        np.testing.assert_array_almost_equal(
            get_state_vector(output),
            get_state_vector(1j * self.ket10))

    def test_toffoli_gate(self):
        """Toffoli on |110> (controls=0,1; target=2) must give
    |111>."""
        toffoli_op = TOFFOLI(N=3, controls=[0, 1], target=2)
        output     = apply(toffoli_op, qt.tensor(self.ket1, self.
    ket1, self.ket0))
        np.testing.assert_array_almost_equal(
            get_state_vector(output),
            get_state_vector(qt.tensor(self.ket1, self.ket1, self
    .ket1)))
```

Listing 14: Selected unit tests (`gate_test.py`)

The tests use `np.testing.assert_array_almost_equal` rather than exact equality to account for floating-point rounding in the matrix exponential routines inside QuTiP.

8 Software Architecture Overview

The five modules form a layered architecture:

Module	Role	Depends on
`quantum_gates.py`	Gate definitions, apply, utilities	`qutip`, `qutip_qip`
`bloch_sphere.py`	Bloch sphere visualisation	`qutip`, `matplotlib`
`simulate.py`	Single-qubit experiment driver	`quantum_gates`, `bloch_sphere`
`bell_state.py`	Two-qubit Bell state circuit	`quantum_gates`
`gate_test.py`	Automated unit tests	`quantum_gates`, `unittest`

`quantum_gates.py` is the sole interface to QuTiP and `qutip_qip`; all higher modules import exclusively from it, keeping the physics layer decoupled from the simulation and test logic.

9 Conclusion

The software suite successfully maps the core mathematical structures of quantum computation onto Python objects. Qubit states are QuTiP column vectors, quantum gates

are unitary `Qobj` matrices, and gate application reduces to operator multiplication. The Bloch sphere visualisation provides geometric intuition for single-qubit dynamics, while the Bell state simulation demonstrates that multi-qubit entanglement is reproduced correctly. The automated test suite in `gate_test.py` validates each gate against its analytically known action, providing a reproducible correctness guarantee for the implementation.